

Border Fence Vibration Detection

```
from machine import Pin, I2C
```

```
import utime
```

```
import math
```

```
# =====
```

```
# CONFIGURATION
```

```
# =====
```

```
I2C_ID = 0
```

```
SDA_PIN = 0
```

```
SCL_PIN = 1
```

```
I2C_FREQ = 100000
```

```
ACC_ADDRESS = 0x19
```

```
CTRL_REG1_A = 0x20
```

```
CTRL_REG4_A = 0x23
```

```
OUT_X_L_A = 0x28
```

```
AUTO_INCREMENT = 0x80
```

```
LED_PIN = 15
```

```
VIBRATION_THRESHOLD = 120
```

```
EVENT_COUNT_THRESHOLD = 5
```

```
WINDOW_TIME_MS = 3000
```

```
ALPHA = 0.20
```

```
# =====
```

```
# GLOBAL VARIABLES
```

```
# =====
```

```
x_bias = 0
```

```
y_bias = 0
```

```
z_bias = 0
```

```
filtered_mag = None
```

```
prev_x = None
```

```
prev_y = None
```

```
prev_z = None
```

```
event_counter = 0
```

```
window_start = utime.ticks_ms()
```

```
# =====
```

```
# GPIO
```

```
# =====
```

```
led = Pin(LED_PIN, Pin.OUT)
```

```
# =====
```

```
# I2C
```

```
# =====
```

```
i2c = I2C(
```

```
    I2C_ID,
```

```
    scl=Pin(SCL_PIN),
```

```
    sda=Pin(SDA_PIN),
```

```
    freq=I2C_FREQ
```

)

=====

SAFE WRITE

=====

def write_register(register, value):

retry = 3

while retry:

try:

i2c.writeto_mem(
 ACC_ADDRESS,
 register,
 bytes([value])
)

return True

except OSError:

retry -= 1

utime.sleep_ms(50)

return False

=====

```
# SAFE READ
```

```
# =====
```

```
def read_registers(register, length):
```

```
    retry = 3
```

```
    while retry:
```

```
        try:
```

```
            return i2c.readfrom_mem(
                ACC_ADDRESS,
                register,
                length
            )
```

```
        except OSError:
```

```
            retry -= 1
```

```
            utime.sleep_ms(50)
```

```
    return None
```

```
# =====
```

```
# SENSOR CHECK
```

```
# =====
```

```
def sensor_present():
```

```
return ACC_ADDRESS in i2c.scan()
```

```
# =====
```

```
# INITIALIZE SENSOR
```

```
# =====
```

```
def initialize_sensor():
```

```
    if not sensor_present():
```

```
        raise Exception("Accelerometer Not Found")
```

```
    write_register(CTRL_REG1_A, 0x57)
```

```
    write_register(CTRL_REG4_A, 0x08)
```

```
    utime.sleep_ms(100)
```

```
# =====
```

```
# SIGNED CONVERSION
```

```
# =====
```

```
def signed(value):
```

```
    value >>= 4
```

```
    if value >= 2048:
```

```
        value -= 4096
```

```
    return value
```

```

# =====

# READ XYZ

# =====

def read_accelerometer():

    data = read_registers(
        OUT_X_L_A | AUTO_INCREMENT,
        6
    )

    if data is None:

        return None

    x = (data[1] << 8) | data[0]
    y = (data[3] << 8) | data[2]
    z = (data[5] << 8) | data[4]

    x = signed(x)
    y = signed(y)
    z = signed(z)

    return x, y, z

# =====

# CALIBRATION

# =====

def calibrate(samples=300):

```

```
global x_bias
```

```
global y_bias
```

```
global z_bias
```

```
sx = 0
```

```
sy = 0
```

```
sz = 0
```

```
count = 0
```

```
while count < samples:
```

```
    values = read_accelerometer()
```

```
    if values is None:
```

```
        continue
```

```
    x, y, z = values
```

```
    sx += x
```

```
    sy += y
```

```
    sz += z
```

```
    count += 1
```

```
    utime.sleep_ms(10)
```

```
x_bias = sx / samples
```

```
y_bias = sy / samples
```

```
z_bias = (sz / samples) - 1024
```

```

# =====

# CALIBRATED XYZ

# =====

def get_xyz():

    values = read_accelerometer()

    if values is None:

        return None

    x, y, z = values

    x -= x_bias
    y -= y_bias
    z -= z_bias

    return x, y, z
# =====

# VIBRATION MAGNITUDE

# =====

def vibration_magnitude(x, y, z):

    global prev_x
    global prev_y
    global prev_z

    if prev_x is None:

```



```
prev_x = x
```

```
prev_y = y
```

```
prev_z = z
```

```
return 0
```

```
dx = x - prev_x
```

```
dy = y - prev_y
```

```
dz = z - prev_z
```

```
prev_x = x
```

```
prev_y = y
```

```
prev_z = z
```

```
magnitude = math.sqrt(
```

```
    dx * dx +
```

```
    dy * dy +
```

```
    dz * dz
```

```
)
```

```
return magnitude
```

```
# =====
```

```
# LOW PASS FILTER
```

```
# =====
```

```
def low_pass(value):
```

```
    global filtered_mag
```

```
if filtered_mag is None:
```

```
    filtered_mag = value
```

```
    return filtered_mag
```

```
filtered_mag = (
```

```
    ALPHA * value +
```

```
    (1 - ALPHA) * filtered_mag
```

```
)
```

```
return filtered_mag
```

```
# =====
```

```
# VIBRATION DETECTION
```

```
# =====
```

```
def detect_vibration(magnitude):
```

```
    if magnitude >= VIBRATION_THRESHOLD:
```

```
        return True
```

```
    return False
```

```
# =====
```

```
# EVENT WINDOW CHECK
```

```
# =====
```

```
def event_window(vibration):
```

```
global event_counter
```

```
global window_start
```

```
current = utime.ticks_ms()
```

```
if vibration:
```

```
    event_counter += 1
```

```
if utime.ticks_diff(current, window_start) >= WINDOW_TIME_MS:
```

```
    alarm = False
```

```
    if event_counter >= EVENT_COUNT_THRESHOLD:
```

```
        alarm = True
```

```
    event_counter = 0
```

```
    window_start = current
```

```
    return alarm
```

```
return False
```

```
# =====
```

```
# LED / BUZZER
```

```
# =====
```

```
def alarm(status):
```

```
    if status:
```

```
led.value(1)
```

```
else:
```

```
led.value(0)
```

```
# =====
```

```
# DISPLAY
```

```
# =====
```

```
def display_data(
```

```
    x,
```

```
    y,
```

```
    z,
```

```
    magnitude,
```

```
    vibration,
```

```
    alarm_status,
```

```
    count
```

```
):
```

```
    print("-----")
```

```
    print("X :", round(x, 2))
```

```
    print("Y :", round(y, 2))
```

```
    print("Z :", round(z, 2))
```

```
    print("Magnitude :", round(magnitude, 2))
```

```
    print("Vibration :", vibration)
```

```
print("Event Count :", count)
```

```
print("Alarm :", alarm_status)
```

```
# =====
```

```
# MAIN PROGRAM
```

```
# =====
```

```
def main():
```

```
    global event_counter
```

```
    print("-----")
```

```
    print("Border Fence Vibration Detection System")
```

```
    print("-----")
```

```
    initialize_sensor()
```

```
    print("Sensor Initialized")
```

```
    print("Calibrating...")
```

```
    calibrate(300)
```

```
    print("Calibration Complete")
```

```
    utime.sleep_ms(1000)
```

```
    while True:
```

```
        try:
```

```
values = get_xyz()
```

```
if values is None:
```

```
    continue
```

```
x, y, z = values
```

```
magnitude = vibration_magnitude(
```

```
    x,
```

```
    y,
```

```
    z
```

```
)
```

```
magnitude = low_pass(
```

```
    magnitude
```

```
)
```

```
vibration = detect_vibration(
```

```
    magnitude
```

```
)
```

```
alarm_status = event_window(
```

```
    vibration
```

```
)
```

```
alarm(
```

```
    alarm_status
```

```
)
```

```
display_data(
```

```
x,  
y,  
z,  
magnitude,  
vibration,  
alarm_status,  
event_counter  
)
```

```
utime.sleep_ms(50)
```

```
except KeyboardInterrupt:
```

```
led.value(0)
```

```
print("Program Stopped")
```

```
break
```

```
except OSError:
```

```
print("I2C Communication Error")
```

```
utime.sleep_ms(200)
```

```
except Exception as e:
```

```
print("Error:", e)
```

```
utime.sleep_ms(500)
```

```
# =====

# START

# =====

if __name__ == "__main__":

    main()
```

The **Border Fence Vibration Detection System** is an embedded security application developed using the **Raspberry Pi Pico** and the **LSM303DLHC accelerometer**. The purpose of this project is to continuously monitor vibrations occurring on a border fence or any secured physical barrier. The LSM303DLHC sensor is mounted firmly on the fence and connected to the Raspberry Pi Pico through the **I2C communication protocol**. When the program starts, the Pico initializes the sensor and performs calibration to remove sensor bias while the fence remains stationary. It then continuously reads the X, Y, and Z acceleration values from the accelerometer. These raw values are converted into signed values and calibrated to improve accuracy. The program calculates the vibration magnitude by comparing the current sensor readings with the previous readings. A low-pass filter is applied to remove unwanted noise caused by wind or minor environmental disturbances. If the vibration magnitude exceeds a predefined threshold multiple times within a specified time window, the system identifies it as a significant disturbance instead of a random vibration. Once this condition is met, the Raspberry Pi Pico activates an alarm by turning on an LED or buzzer and displays the alert information in the Thonny serial console. The entire monitoring process runs continuously in real time and includes basic error handling to recover from temporary I2C communication errors without stopping the program.

How to Give Input to the System

This project does not require any keyboard, mouse, or manual data entry because the **LSM303DLHC sensor itself acts as the input device**. After uploading the MicroPython program to the Raspberry Pi Pico using Thonny, power the Pico and keep the sensor completely still during the calibration process. Once calibration is completed, securely attach the sensor to a fence, metal rod, gate, or any object you want to monitor. To generate input, simply create vibrations by gently tapping the fence, shaking it, knocking on it, or moving the attached object. The accelerometer senses these physical vibrations as changes in acceleration along the X, Y, and Z axes. The program automatically processes these changes, calculates the vibration magnitude, filters out small disturbances, and checks whether the vibration exceeds the configured threshold. If repeated strong vibrations occur within the specified time period, the alarm is activated automatically. Therefore, the only input required for this project is **physical vibration or movement**, making the system completely autonomous and suitable for offline operation.

Uses of This Application

The Border Fence Vibration Detection System can be used in many real-world security and monitoring applications. In the defense sector, it can be installed on border fences, military camp perimeters, ammunition depots, communication towers, temporary security barriers, and restricted areas to detect unauthorized attempts to climb, cut, or shake the fence. It provides an early warning

system that can alert security personnel before an intruder enters the protected area. Beyond defense, the same system can be used for securing industrial plants, warehouses, power stations, railway tracks, airport boundaries, prisons, construction sites, farms, and private properties. It can also monitor valuable equipment, storage containers, and restricted rooms where physical disturbances must be detected immediately. Since the system works completely offline, consumes very little power, and uses inexpensive hardware, it is suitable for remote locations where internet connectivity is unavailable or where a simple, reliable, and cost-effective vibration monitoring solution is required.

: Drone Orientation Monitoring

```
from machine import Pin, I2C
```

```
import utime
```

```
import math
```

```
# =====
```

```
# CONFIGURATION
```

```
# =====
```

```
I2C_ID = 0
```

```
SDA_PIN = 0
```

```
SCL_PIN = 1
```

```
I2C_FREQ = 100000
```

```
ACC_ADDRESS = 0x19
```

```
MAG_ADDRESS = 0x1E
```

```
CTRL_REG1_A = 0x20
```

```
CTRL_REG4_A = 0x23
```

```
CRA_REG_M = 0x00
```

```
CRB_REG_M = 0x01
```

```
MR_REG_M = 0x02
```

OUT_X_L_A = 0x28

AUTO_INCREMENT = 0x80

MAG_OUT_X_H = 0x03

ALPHA = 0.20

=====

GLOBAL VARIABLES

=====

x_bias = 0

y_bias = 0

z_bias = 0

mx_bias = 0

my_bias = 0

mz_bias = 0

filtered_roll = None

filtered_pitch = None

filtered_heading = None

=====

I2C

=====

i2c = I2C(

 I2C_ID,

 scl=Pin(SCL_PIN),

```

    sda=Pin(SDA_PIN),
    freq=I2C_FREQ
)

# =====
# SAFE WRITE
# =====

def write_register(device, register, value):

    retry = 3

    while retry:

        try:

            i2c.writeto_mem(
                device,
                register,
                bytes([value])
            )

            return True

        except OSError:

            retry -= 1

            utime.sleep_ms(50)

    return False

```

```

# =====

# SAFE READ

# =====

def read_registers(device, register, length):

    retry = 3

    while retry:

        try:

            return i2c.readfrom_mem(
                device,
                register,
                length
            )

        except OSError:

            retry -= 1

            utime.sleep_ms(50)

    return None

# =====

# SENSOR CHECK

# =====

```

```

def sensors_present():

    devices = i2c.scan()

    return (
        ACC_ADDRESS in devices and
        MAG_ADDRESS in devices
    )

# =====
# INITIALIZE SENSORS
# =====

def initialize_sensor():

    if not sensors_present():

        raise Exception("Sensor Not Found")

    write_register(
        ACC_ADDRESS,
        CTRL_REG1_A,
        0x57
    )

    write_register(
        ACC_ADDRESS,
        CTRL_REG4_A,
        0x08
    )

```

```
write_register(  
    MAG_ADDRESS,  
    CRA_REG_M,  
    0x14  
)
```

```
write_register(  
    MAG_ADDRESS,  
    CRB_REG_M,  
    0x20  
)
```

```
write_register(  
    MAG_ADDRESS,  
    MR_REG_M,  
    0x00  
)
```

```
utime.sleep_ms(100)
```

```
# =====  
# SIGNED  
# =====
```

```
def signed12(value):
```

```
    value >>= 4
```

```
    if value >= 2048:
```

```
        value -= 4096
```

```
    return value
```

```
def signed16(value):
```

```
    if value > 32767:
```

```
        value -= 65536
```

```
    return value
```

```
# =====
```

```
# READ ACCELEROMETER
```

```
# =====
```

```
def read_accelerometer():
```

```
    data = read_registers(
        ACC_ADDRESS,
        OUT_X_L_A | AUTO_INCREMENT,
        6
    )
```

```
    if data is None:
```

```
        return None
```

```
    x = signed12((data[1] << 8) | data[0])
```

```
    y = signed12((data[3] << 8) | data[2])
```

```
    z = signed12((data[5] << 8) | data[4])
```

```
return x, y, z
```

```
# =====
```

```
# READ MAGNETOMETER
```

```
# =====
```

```
def read_magnetometer():
```

```
    data = read_registers(
```

```
        MAG_ADDRESS,
```

```
        MAG_OUT_X_H,
```

```
        6
```

```
    )
```

```
    if data is None:
```

```
        return None
```

```
    mx = signed16((data[0] << 8) | data[1])
```

```
    mz = signed16((data[2] << 8) | data[3])
```

```
    my = signed16((data[4] << 8) | data[5])
```

```
    return mx, my, mz
```

```
# =====
```

```
# CALIBRATION
```

```
# =====
```

```
def calibrate(samples=300):
```

```
    global x_bias
```



```
global y_bias
```

```
global z_bias
```

```
sx = sy = sz = 0
```

```
count = 0
```

```
while count < samples:
```

```
    values = read_accelerometer()
```

```
    if values is None:
```

```
        continue
```

```
    x, y, z = values
```

```
    sx += x
```

```
    sy += y
```

```
    sz += z
```

```
    count += 1
```

```
    utime.sleep_ms(10)
```

```
x_bias = sx / samples
```

```
y_bias = sy / samples
```

```
z_bias = (sz / samples) - 1024
```

```
# =====
```

```
# GET XYZ
```

```
# =====
```

```
def get_xyz():
```

```
    values = read_accelerometer()
```

```
    if values is None:
```

```
        return None
```

```
    x, y, z = values
```

```
    x -= x_bias
```

```
    y -= y_bias
```

```
    z -= z_bias
```

```
    return x, y, z
```

```
# =====
```

```
# ROLL AND PITCH
```

```
# =====
```

```
def calculate_angles(x, y, z):
```

```
    roll = math.atan2(y, z)
```

```
    pitch = math.atan2(
```

```
        -x,
```

```
        math.sqrt((y * y) + (z * z))
```

```
    )
```

```
    roll = math.degrees(roll)
```

```
pitch = math.degrees(pitch)
```

```
return roll, pitch
```

```
# =====
```

```
# TILT COMPENSATED HEADING
```

```
# =====
```

```
def calculate_heading(mx, my, mz, roll, pitch):
```

```
    roll = math.radians(roll)
```

```
    pitch = math.radians(pitch)
```

```
    xh = (
```

```
        mx * math.cos(pitch)
```

```
        +
```

```
        mz * math.sin(pitch)
```

```
    )
```

```
    yh = (
```

```
        mx * math.sin(roll) * math.sin(pitch)
```

```
        +
```

```
        my * math.cos(roll)
```

```
        -
```

```
        mz * math.sin(roll) * math.cos(pitch)
```

```
    )
```

```
    heading = math.degrees(
```

```
        math.atan2(
```

```
            yh,
```

```
        xh
    )
)
```

```
if heading < 0:
```

```
    heading += 360
```

```
return heading
```

```
# =====
# LOW PASS FILTER
# =====
```

```
def low_pass_filter(roll, pitch, heading):
```

```
    global filtered_roll
```

```
    global filtered_pitch
```

```
    global filtered_heading
```

```
if filtered_roll is None:
```

```
    filtered_roll = roll
```

```
    filtered_pitch = pitch
```

```
    filtered_heading = heading
```

```
return (
```

```
    filtered_roll,
```

```
    filtered_pitch,
```

```
    filtered_heading
```

)

```
filtered_roll = (  
    ALPHA * roll +  
    (1 - ALPHA) * filtered_roll  
)
```

```
filtered_pitch = (  
    ALPHA * pitch +  
    (1 - ALPHA) * filtered_pitch  
)
```

```
filtered_heading = (  
    ALPHA * heading +  
    (1 - ALPHA) * filtered_heading  
)
```

```
return (  
    filtered_roll,  
    filtered_pitch,  
    filtered_heading  
)
```

```
# =====  
# COMPASS DIRECTION  
# =====
```

```
def compass_direction(heading):
```

```
    if heading >= 337.5 or heading < 22.5:
```

```
    return "North"
```

```
elif heading < 67.5:
```

```
    return "North-East"
```

```
elif heading < 112.5:
```

```
    return "East"
```

```
elif heading < 157.5:
```

```
    return "South-East"
```

```
elif heading < 202.5:
```

```
    return "South"
```

```
elif heading < 247.5:
```

```
    return "South-West"
```

```
elif heading < 292.5:
```

```
    return "West"
```

```
else:
```

```
    return "North-West"
```

```
# =====
```

```
# DISPLAY
```

```
# =====
```

```
def display_data(
```

```
    x,
```

```
    y,
```

```
z,  
mx,  
my,  
mz,  
roll,  
pitch,  
heading,  
direction  
):
```

```
print("-----")
```

```
print(  
    "ACC X :",  
    round(x, 2)  
)
```

```
print(  
    "ACC Y :",  
    round(y, 2)  
)
```

```
print(  
    "ACC Z :",  
    round(z, 2)  
)
```

```
print(  
    "MAG X :",  
    mx  
)
```

```
print(  
    "MAG Y :",  
    my  
)
```

```
print(  
    "MAG Z :",  
    mz  
)
```

```
print(  
    "ROLL :",  
    round(roll, 2)  
)
```

```
print(  
    "PITCH :",  
    round(pitch, 2)  
)
```

```
print(  
    "HEAD :",  
    round(heading, 2)  
)
```

```
print(  
    "DIR :",  
    direction  
)
```

```
# =====
```



```
# MAIN PROGRAM
```

```
# =====
```

```
def main():
```

```
    print("-----")
```

```
    print("Drone Orientation Monitoring System")
```

```
    print("-----")
```

```
    initialize_sensor()
```

```
    print("Sensors Initialized")
```

```
    print("Calibrating Accelerometer...")
```

```
    calibrate(300)
```

```
    print("Calibration Complete")
```

```
    utime.sleep_ms(1000)
```

```
    while True:
```

```
        try:
```

```
            acc = get_xyz()
```

```
            mag = read_magnetometer()
```

```
            if acc is None or mag is None:
```

```
continue
```

```
x, y, z = acc
```

```
mx, my, mz = mag
```

```
roll, pitch = calculate_angles(
```

```
    x,
```

```
    y,
```

```
    z
```

```
)
```

```
heading = calculate_heading(
```

```
    mx,
```

```
    my,
```

```
    mz,
```

```
    roll,
```

```
    pitch
```

```
)
```

```
roll, pitch, heading = low_pass_filter(
```

```
    roll,
```

```
    pitch,
```

```
    heading
```

```
)
```

```
direction = compass_direction(
```

```
    heading
```

```
)
```

```
display_data(
```

```
x,  
y,  
z,  
mx,  
my,  
mz,  
roll,  
pitch,  
heading,  
direction  
)
```

```
utime.sleep_ms(100)
```

```
except KeyboardInterrupt:
```

```
    print("Program Stopped")
```

```
    break
```

```
except OSError:
```

```
    print("I2C Communication Error")
```

```
    utime.sleep_ms(200)
```

```
except Exception as e:
```

```
    print("Error :", e)
```

```
    utime.sleep_ms(500)
```

```
# =====

# START PROGRAM

# =====

if __name__ == "__main__":

    main()
```

Project Explanation

The **Drone Orientation Monitoring System** is an embedded application developed using the **Raspberry Pi Pico** and the **LSM303DLHC sensor**, which contains both an accelerometer and a magnetometer. The purpose of this project is to continuously monitor the orientation and heading of a drone or any moving platform in real time. The Raspberry Pi Pico communicates with the sensor using the **I2C protocol** and initializes both sensing modules when the program starts. The accelerometer measures acceleration along the X, Y, and Z axes, while the magnetometer measures the Earth's magnetic field. The program first calibrates the accelerometer to remove sensor bias and improve accuracy. It then continuously reads sensor data, converts the raw values into signed values, and calculates the **roll** and **pitch** angles to determine the drone's tilt. Using the magnetometer together with the roll and pitch values, the program calculates a **tilt-compensated heading**, which provides the drone's direction even when it is tilted. A low-pass filter is applied to reduce sensor noise and make the output stable. Finally, the heading is converted into compass directions such as North, East, South, West, and intermediate directions, and all the information is displayed continuously in the Thonny serial console. The entire process runs offline without requiring an internet connection, making it suitable for embedded and autonomous applications.

How to Give Input to the System

The input for this project is provided entirely through the **physical movement and rotation of the sensor**. After uploading the MicroPython program to the Raspberry Pi Pico using Thonny, power the board and keep the sensor stationary during the calibration process. Once calibration is completed, mount the LSM303DLHC sensor securely on a drone, robotic platform, or simply hold it in your hand for testing. Rotate the sensor left or right to change its heading, tilt it forward or backward to change the pitch angle, and tilt it sideways to change the roll angle. The accelerometer detects changes in orientation, while the magnetometer senses the Earth's magnetic field to determine the direction in which the sensor is pointing. The Raspberry Pi Pico continuously processes these sensor readings, calculates the roll, pitch, and heading values, and displays the updated orientation and compass direction in real time. No buttons, keyboard, or manual data entry are required because the sensor itself acts as the input device.

Uses of This Application

The Drone Orientation Monitoring System has many practical applications in defense, aerospace, robotics, and navigation. In the defense sector, it can be used to monitor the orientation and heading of unmanned aerial vehicles (UAVs), surveillance drones, reconnaissance systems, autonomous

ground vehicles, and portable navigation equipment. It helps operators understand the drone's position and direction during surveillance, mapping, border patrol, and search-and-rescue missions. The same system can also be integrated into robotic vehicles, self-balancing robots, camera stabilization systems, autonomous delivery robots, marine vehicles, and navigation devices that require accurate orientation information. It is useful in educational laboratories for learning embedded systems, sensor interfacing, and navigation algorithms. Since the project operates completely offline, consumes very little power, and uses low-cost hardware, it is suitable for remote environments where reliable real-time orientation monitoring is required without dependence on external communication networks.

Electronic Navigation Aid (Compass + Tilt Compensation)

For Project 4 (Electronic Navigation Aid)

Do not write a completely new code.

Instead:

1. **Copy the Drone Orientation Monitoring code.**
2. Rename the project to:
 - **Electronic Navigation Aid (Compass + Tilt Compensation)**
3. Change only the application logic and output.

For example, instead of printing:

Roll : 5°
Pitch : -3°
Heading : 90°
Direction : East

print:

Electronic Navigation Aid

Current Heading : 90°

Direction : East

Navigation Status : Moving Towards East

Or, if you want to navigate to North:

Python

Run

TARGET_HEADING = 0

if heading > 10:

```
    print("Turn Left")

elif heading < -10:
    print("Turn Right")

else:
    print("Move Forward")

Or using 0–360°:

Python

Run

error = TARGET_HEADING - heading

if abs(error) < 5:
    print("Move Forward")

elif error > 0:
    print("Turn Left")

else:
    print("Turn Right")
```

This makes it a different application

Drone Orientation Monitoring

- Monitors the drone's roll, pitch, and heading.
- Purpose: Observe orientation.

Electronic Navigation Aid

- Uses the same sensor data.
- Guides the user to face a desired direction.
- Purpose: Navigation assistance.

The underlying sensor code remains the same because the hardware is the same, but the application logic is different.

Project Explanation

The **Electronic Navigation Aid (Compass + Tilt Compensation)** is an embedded system developed using the **Raspberry Pi Pico** and the **LSM303DLHC sensor**, which combines an accelerometer and a magnetometer to provide accurate direction and orientation information. The Raspberry Pi Pico communicates with the sensor through the **I2C protocol** and continuously reads acceleration and magnetic field data. At the beginning of the program, the accelerometer is calibrated to remove sensor bias and improve measurement accuracy. The accelerometer data is then used to calculate

the **roll** and **pitch** angles, while the magnetometer measures the Earth's magnetic field to determine the heading. Since the sensor may be tilted during operation, the program performs **tilt compensation**, which corrects the heading based on the calculated roll and pitch values. This ensures that the compass direction remains accurate even when the device is not held perfectly level. A low-pass filter is applied to reduce sensor noise and stabilize the output. Finally, the Raspberry Pi Pico displays the heading in degrees along with the corresponding compass direction such as North, North-East, East, South-East, South, South-West, West, or North-West. The entire system operates completely offline and provides continuous real-time navigation information without requiring internet connectivity or GPS.

How to Give Input to the System

The input to this project is provided entirely through the **physical movement and orientation of the LSM303DLHC sensor**. After uploading the MicroPython program to the Raspberry Pi Pico using Thonny, power the board and keep the sensor stationary during the calibration process. Once calibration is complete, hold the sensor in your hand or mount it on a portable device. Rotate the sensor clockwise or counterclockwise to change its heading, tilt it forward and backward to change the pitch angle, or tilt it left and right to change the roll angle. As the sensor moves, the accelerometer measures changes in orientation while the magnetometer detects the Earth's magnetic field. The Raspberry Pi Pico continuously processes these values, performs tilt compensation, calculates the corrected heading, and displays the updated compass direction in the Thonny serial monitor. No keyboard, buttons, or manual data entry are required because the sensor itself provides all the necessary input through its physical movement.

Uses of This Application

The **Electronic Navigation Aid** has many practical applications in defense, navigation, robotics, and outdoor activities. In the defense sector, it can assist soldiers in determining their heading during patrols, reconnaissance missions, and navigation in areas where GPS signals are weak or unavailable. It can also be integrated into portable navigation devices, military vehicles, unmanned ground vehicles, and field equipment to provide accurate directional information. Beyond defense, the same system can be used in robotic navigation, autonomous vehicles, marine navigation, hiking and trekking devices, digital compasses, educational laboratories, and embedded systems research. It is also useful in rescue operations, surveying, and exploration where knowing the correct direction is important. Since the project is compact, low-cost, energy-efficient, and works completely offline, it provides a reliable navigation solution for environments where continuous and accurate compass information is required without depending on external communication networks or internet connectivity.